

ANALİZ VE TASARIM RAPORU

Akıllı Tedarik ve Lojistik Yönetim Sistemi

Nesneye Dayalı Analiz ve Tasarım Dersi - Dönem Sonu Projesi

Ad	Umut
Soyad	Uslu
Numara	B221210012
Sınıf	1-A
Tarih	2025 - 2026

1. Giriş ve Proje Amacı

Bu rapor, bir e-ticaret lojistik merkezinin yazılım ihtiyaçlarını karşılayan 'Akıllı Tedarik ve Lojistik Yönetim Sistemi' projesinin analiz ve tasarım aşamalarını belgeleyen dokümandır.

Sistem; stok takibi, sipariş yönetimi, kargo stratejileri ve rol tabanlı yetkilendirme gibi temel iş süreçlerini kapsamaktadır. Yazılım, MVC mimarisi, SOLID prensipleri ve çeşitli Tasarım Desenleri kullanılarak C# Windows Forms platformunda geliştirilmiştir.

2. Gereksinim Analizi

2.1 Fonksiyonel Gereksinimler

No	Modül	Gereksinim
FR-01	Ürün Yönetimi	Sistem hem basit (kalem) hem de bileşik ürünleri (bilgisayar kasası - RAM, CPU içeren) desteklemelidir.
FR-02	Stok Takibi	Ürünün stoğu belirlenen eşik değerinin altına düştüğünde Satın Alma birimine e-posta, Depo sorumlusuna sistem içi bildirim gönderilmelidir.
FR-03	Sipariş Akışı	Sipariş; Beklemede → Onaylandı → Hazırlanıyor → Kargoda → Teslim Edildi aşamalarından geçmelidir.
FR-04	İade Yönetimi	Kargoda veya Teslim Edildi aşamasındaki sipariş iade sürecine alınabilmelidir. Kargodaki sipariş iptal edilememeli.
FR-05	Ödeme	Sistem; Kredi Kartı, Havale ve ileride eklenebilecek Kripto Ödeme yöntemlerini desteklemelidir.
FR-06	Kargo	Aras, Yurtiçi ve GlobalExpres firmalarının farklı API'leri tek standart arayüz üzerinden kullanılabilmelidir.
FR-07	Kargo Ücret	Kargo ücreti; ağırlık, mesafe ve ek hizmetlere (sigorta, kırılman koruma) göre dinamik hesaplanmalıdır.
FR-08	Yetkilendirme	Kullanıcı rolüne göre (Admin, Depo Görevlisi, Kurye, Müşteri) görüntü ve işlem yetkileri kısıtlanmalıdır.
FR-09	Loglama	Her kritik işlem (stok değişimi, ödeme onayı, durum güncelleme) dosyaya loglanmalıdır. Loglama nesnesi sistemde tek olmalıdır.

2.2 Fonksiyonel Olmayan Gereksinimler

Kategori	Gereksinim
Genişletilebilirlik	Yeni kargo firması, ödeme yöntemi veya bildirim kanalı eklemek için mevcut koda dokunulmamalıdır.
Sürdürülebilirlik	Kod SOLID prensiplerine uyumlu olmalı, her sınıfın tek sorumluluğu bulunmalıdır.
Güvenlik	Rol tabanlı erişim kontrolü sağlanmalı, her işlem öncesinde yetki doğrulanmalıdır.
Kalıcılık	Veriler JSON formatında dosyaya kaydedilmeli, uygulama yeniden başlatıldığında veriler korunmalıdır.
Test Edilebilirlik	Kritik iş mantığı birim testleri ile test edilebilir olmalıdır.

3. Kullanıcı Roller ve Yetkiler

İşlem	Admin	Depo Görevlisi	Kurye	Müşteri
Kullanıcı yönetimi	✓	X	X	X
Ürün ekleme/silme	✓	X	X	X
Stok güncelleme	✓	✓	X	X
Sipariş oluşturma	✓	X	X	✓
Sipariş durumu ilet	✓	✓	✓	X
Kargo atama	✓	✓	X	X
Teslim işaretleme	✓	X	✓	X
Sipariş iptal/iade	✓	X	X	✓
Log görüntüleme	✓	X	X	X

4. MVC Mimarisi ve Katman Yapısı

Proje, Model-View-Controller (MVC) mimarisi kullanılarak geliştirilmiştir. Bu mimari sayesinde iş mantığı, veri erişimi ve kullanıcı arayüzü birbirinden bağımsız katmanlara ayrılmıştır.

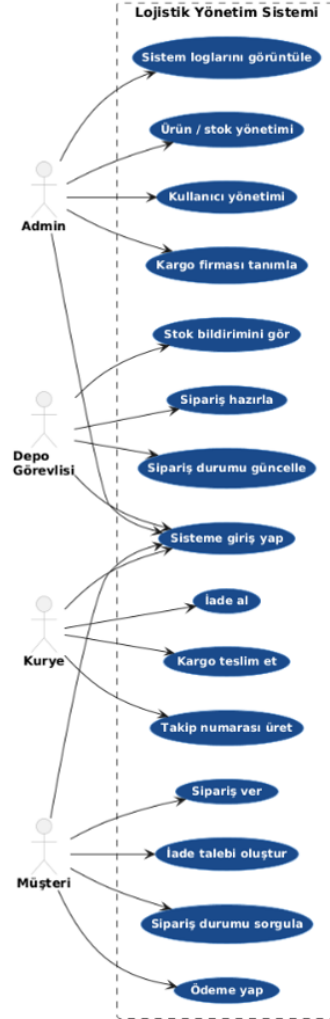
Katman	Klasör / Dosyalar	Sorumluluk
Model	Models/Entities, Models/Enums, Models/Patterns, Models/Repositories	Veri yapıları, iş kuralları, tasarım desenleri ve JSON okuma/yazma işlemleri. View ve Controller'dan bağımsızdır.
View	Views/ (LoginForm, MainForm, SiparisForm, StokForm, KargoForm, KullaniciForm, LogForm)	Kullanıcı arayüzü. Windows Forms ile gerçekleştirilmiştir. İş mantığı içermez, sadece Controller'lara çağrıda bulunur.
Controller	Controllers/ (SiparisController, StokController, KargoController, KullaniciController)	İş mantığının yönetildiği katman. View ile Model arasında aracı görevi üstlenir, yetki kontrollerini yapar.

5. UML Diyagramları

Aşağıdaki UML diyagramları sistemin analiz ve tasarım aşamasında hazırlanmıştır. Kodlama bu diyagramlara dayanılarak gerçekleştirilmiştir.

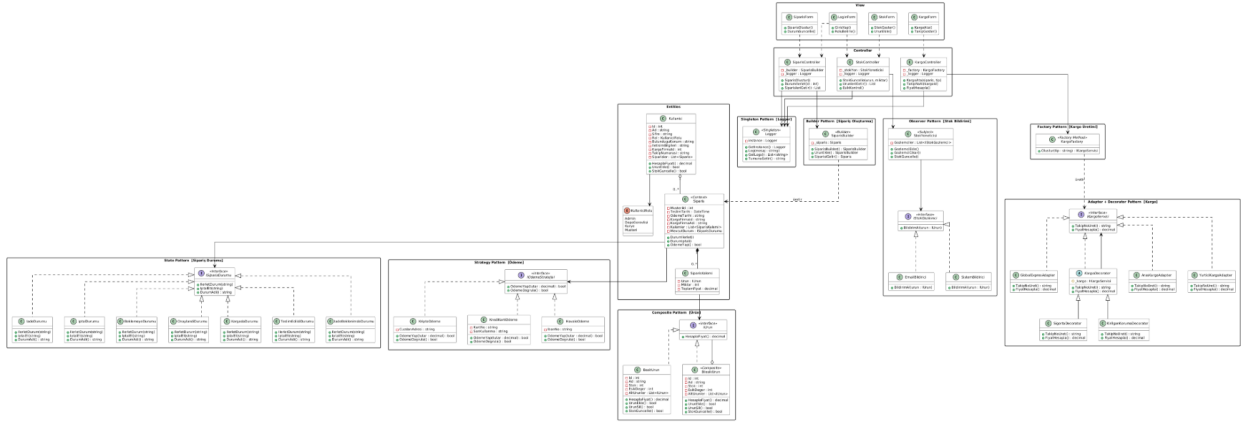
5.1 Use Case Diyagramı

Use Case diyagramı sistemdeki 4 aktörü (Admin, Depo Görevlisi, Kurye, Müşteri) ve bu aktörlerin gerçekleştirebildiği 15 kullanım durumunu göstermektedir. Her rol sadece yetkili olduğu kullanım durumlarıyla ilişkilendirilmiştir.



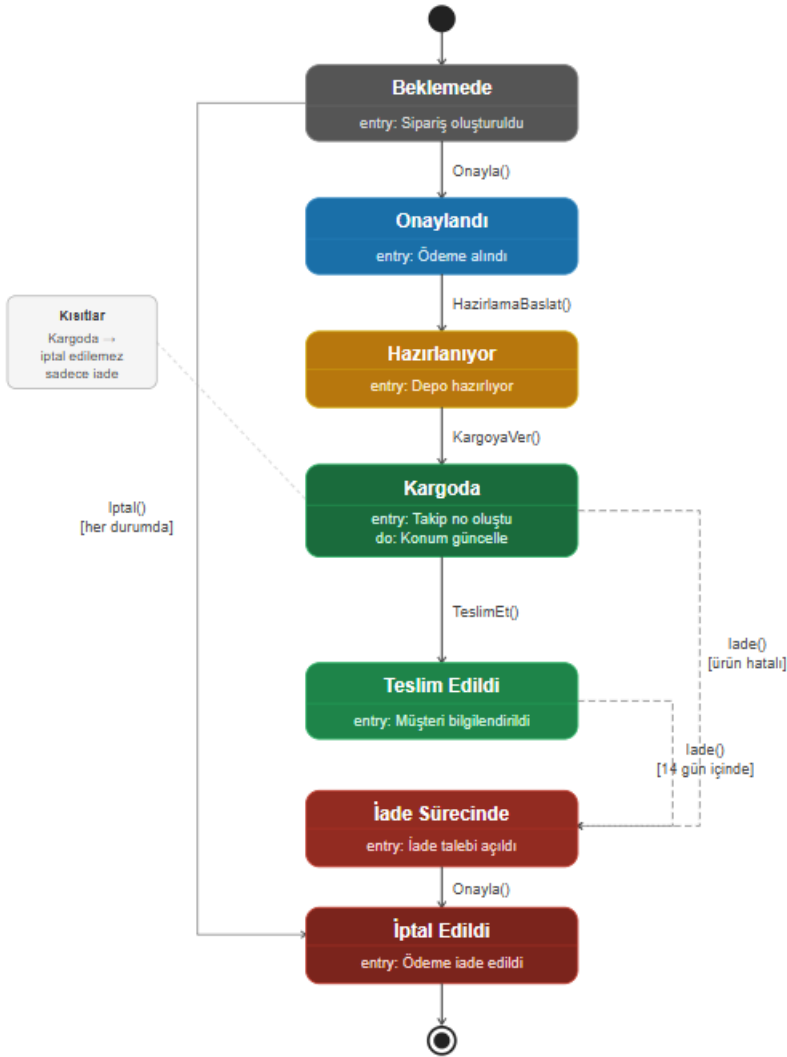
5.2 Sınıf Diyagramı

Sınıf diyagramı MVC katmanlarını, tüm Entity sınıflarını ve 8 tasarım desenini içermektedir. Sınıflar arası ilişkiler (kalıtım, implementasyon, bağımlılık) gösterilmiştir.



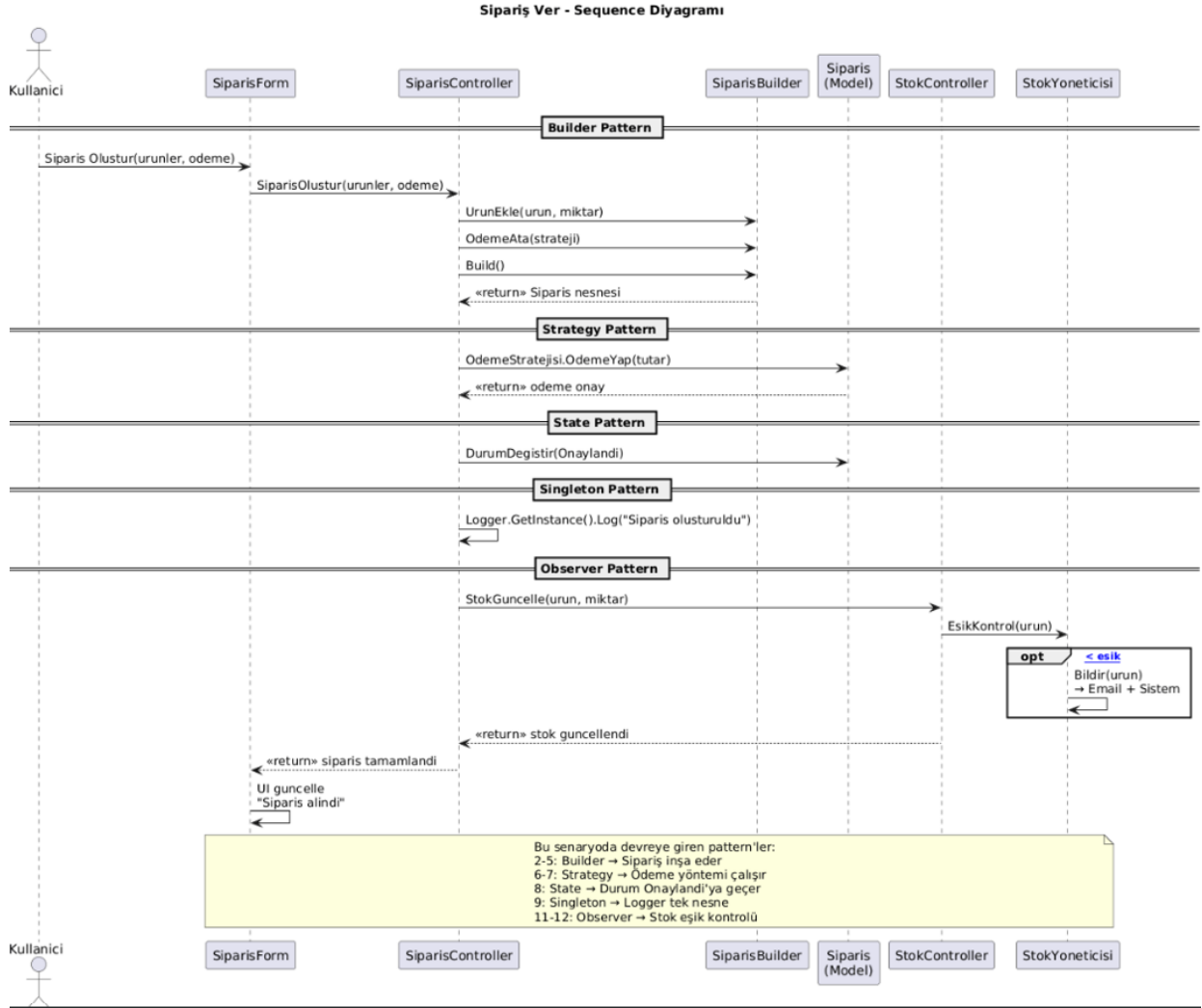
5.3 Durum (State) Diyagramı

Siparişin hayat döngüsü boyunca geçebileceği tüm durumlar ve geçiş koşulları gösterilmiştir. Beklemede → Onaylandı → Hazırlanıyor → Kargoda → Teslim Edildi ana akışının yanı sıra İptal ve İade geçişleri de belgelenmiştir.



5.4 Sequence Diyagramı — Sipariş Ver Senaryosu

Bir müşterinin sipariş vermesi senaryosunda SiparisForm, SiparisController, SiparisBuilder, Siparis (Model), StokController ve StokYoneticisi arasındaki mesaj akışı adım adım gösterilmiştir. Builder, State, Strategy, Observer ve Singleton desenlerinin tek bir senaryoda nasıl zincirleme çalıştığını ortaya koymaktadır.



6. Temel Sistem Akışları

6.1 Sipariş Akışı

Adım	Durum	Sorumlu Rol	Açıklama
1	Beklemede	Müşteri / Admin	Müşteri sepete ürün ekler, ödeme yöntemini seçer, siparişi tamamlar.
2	Onaylandı	Admin / Depo Görevlisi	Ödeme alınır, sipariş sisteme onaylanır.
3	Hazırlanıyor	Depo Görevlisi	Depo görevlisi paketi hazırlar.
4	Kargoda	Depo Görevlisi	Kargo firması atanır, takip numarası üretilir.
5	Teslim Edildi	Kurye	Kurye teslim eder, sipariş kapatılır.
*	İade Sürecinde	Müşteri / Admin	Kargoda veya Teslim Edildi aşamasında iade başlatılabilir.
*	İptal Edildi	Müşteri / Admin	Sadece Beklemede, Onaylandı ve Hazırlanıyor durumlarında iptal edilebilir.

6.2 Stok Bildirim Akışı

Adım	İşlem	Açıklama
1	Stok Düşümü	Sipariş verildiğinde veya stok manuel güncellendiğinde StokController.StokDus() çağrısı yapılır.
2	Eşik Kontrolü	StokYoneticisi yeni stok miktarının eşik değerinin altında olup olmadığını kontrol eder.
3	Observer Tetiklenir	Eşik aşıldıysa kayıtlı tüm gözlemciler (EmailBildirici, SistemBildirici) sırayla bildirilir.
4a	E-posta Bildirimi	EmailBildirici, Satın Alma departmanına bildirim gönderir ve loga yazar.
4b	Sistem Bildirimi	SistemBildirici, YeniBildirimGeldi event'ini tetikler. StokForm bu event'e abone olarak uyarı gösterir.

7. Veri Modeli

Sistem verilerini JSON formatında dosyalara kaydeder. Veritabanı kullanılmamaktadır. Üç JSON dosyası ve bir log dosyası kullanılmaktadır.

Dosya	İçerik	Önemli Alanlar
kullanıcılar.json	Kullanıcı listesi	Id, Ad, KullaniciAdi, Sifre, Rol (0=Admin, 1=Depo, 2=Kurye, 3=Müşteri), SiparisIdleri
urunler.json	Ürün kataloğu	Id, Ad, Tip (Basit/Bileşik), BirimFiyat, Stok, EsikDeger, AltUrunler (Bileşik için)
siparisler.json	Sipariş geçmişi	Id, MusteriId, Tarih, MevcutDurumTip, OdemeYontemi, Kalemler, KargoFirmasi, TakipNumarasi
log.txt	Sistem logları	Tarih-saat damgalı kritik işlem kayıtları. Singleton Logger tarafından yönetilir.

8. Tasarım Desenleri Raporu

Bu bölüm projede kullanılan tasarım desenlerini, her desenin hangi problemi çözdüğünü ve neden o desenin seçildiğini açıklamaktadır. Toplam 8 tasarım deseni kullanılmıştır (minimum 6 gerekliydi): 3 Yaratımsal, 2 Yapısal, 3 Davranışsal.

8.1 Kullanılan Desenler Özeti

Yaratımsal (3 Desen)	Yapısal (2 Desen)	Davranışsal (3 Desen)	Toplam
Singleton, Factory Method, Builder	Adapter, Decorator	State, Strategy, Observer	8 Desen (Min. 6 gerekliydi)

8.2 Desen Kararları Detay Tablosu

Süreç / Modül	Karşılaşılan Problem	Seçilen Desen	Neden Bu Desen?
Sipariş Durumu Yönetimi	Siparişin her durumunda farklı işlemler yapılması gerekiyor. if-else zincirleriyle yönetmek kodun büyümesiyle birlikte okunaksız hale gelir.	State Pattern	Her durum (Beklemede, Onaylandı, Kargoda vb.) kendi sınıfına taşınarak hangi işlemin geçerli olduğuna o sınıf karar verir. Yeni durum eklemek için mevcut koda dokunmak gerekmez.
Ödeme Yöntemi Seçimi	Kredi kartı, havale ve kripto gibi farklı ödeme yöntemleri farklı mantık içerir. switch-case ile yönetmek yeni yöntem eklendiğinde mevcut kodu değiştirmeyi zorunlu kılar.	Strategy Pattern	Her ödeme yöntemi IOdemeStratejisi arayüzünü uygulayan ayrı bir sınıf olur. Yeni yöntem eklemek için sadece yeni sınıf yazılır, Controller'a dokunulmaz. Open/Closed prensibine tam uyum sağlanır.
Stok Eşik Bildirimi	Stok eşik değerinin altına düştüğünde birden fazla tarafı (Satın Alma e-postası, Depo sistemi) haberdar etmek gerekiyor.	Observer Pattern	StokYoneticisi (Subject) stok değişikliklerini izler, eşik aşıldıkça kayıtlı tüm gözlemcileri (EmailBildirici, SistemBildirici) otomatik tetikler.
Kargo Firması API Entegrasyonu	Aras, Yurtiçi ve GlobalExpres firmalarının takip kodu üretme ve fiyatlandırma API'leri birbirinden tamamen farklıdır.	Adapter Pattern	Her kargo firması için bir Adapter sınıfı yazılır ve IKargoServisi arayüzüne uyarlanır. Controller hangi firmanın çalıştığını bilmek zorunda kalmaz.
Kargo Ek Hizmet Ücreti	Sigorta, kırılman koruma gibi ek hizmetlerin her kombinasyonu için ayrı sınıf yazmak kombinasyon patlaması yaratır.	Decorator Pattern	Her ek hizmet bir Decorator sınıfı olur ve istenilen sıralarda sarmalanabilir. Fiyat otomatik hesaplanır, mevcut koda dokunulmaz.
Kargo Servisi Üretimi	KargoController hangi firmayı kullanacağına karar verirken doğrudan new ArasKargoAdapter() yazmak Controller'ı sınıfa sıkıca bağlar.	Factory Method	KargoFactory.Olustur('Aras') çağrısı Controller'ın sadece firma adını bilmesini sağlar. Hangi Adapter sınıfının oluşturulduğu Factory içinde gizlenir.
Sipariş Oluşturma Süreci	Sipariş nesnesi oluşturmak karmaşık: müşteri, ürünler, ödeme yöntemi, stok kontrolü gibi adımlar var.	Builder Pattern	SiparisBuilder ile sipariş adım adım inşa edilir: önce müşteri belirlenir, sonra ürünler eklenir, sonra ödeme atanır.

			Build() çağrısıyla tamamlanmış sipariş nesnesi elde edilir.
Sistem Loglama	Sistemin her tarafından loglama yapılması gerekiyor. Her yerde yeni Logger nesnesi oluşturmak dosyaya eşzamanlı yazma çakışmalarına yol açar.	Singleton Pattern	Logger sınıfı constructor'ını private yapar, GetInstance() ile tek örnek dönüşü sağlar. Double-checked locking ile thread-safety sağlanır.

9. SOLID Prensiplerinin Uygulanması

Prensip	Açıklama	Projede Karşılık Gelen Yer
Single Responsibility (SRP)	Her sınıfın tek bir sorumluluğu olmalıdır.	Controller'lar sadece iş mantığını yönetir, Repository'ler sadece JSON okuma/yazmayı, Pattern sınıfları sadece kendi desenlerini uygulamakla sorumludur.
Open/Closed (OCP)	Sınıflar genişletmeye açık, değişime kapalı olmalıdır.	Yeni kargo firması eklemek için sadece yeni Adapter sınıfı yazılır. Yeni ödeme yöntemi eklemek için sadece yeni Strategy sınıfı yazılır.
Liskov Substitution (LSP)	Alt sınıflar, üst sınıfın yerine kullanılabilir.	ArasKargoAdapter, YurticiKargoAdapter ve GlobalExpresAdapter hepsi IKargoServisi arayüzünü uyguluyor. Controller hangisini kullandığını bilmeden çalışır.
Interface Segregation (ISP)	Sınıflar kullanmadıkları metodları implement etmemelidir.	IKargoServisi, IOdemeStratejisi, ISiparisDurumu, IStokGozlemci gibi küçük ve odaklı arayüzler tanımlanmıştır.
Dependency Inversion (DIP)	Üst katmanlar alt katmanlara değil, soyutlamalara bağımlı olmalıdır.	KargoController, IKargoServisi arayüzüne bağımlıdır; ArasKargoAdapter sınıfına değil. SiparisController, IOdemeStratejisi arayüzüne bağımlıdır.

10. Birim Test Özeti

Projenin kritik iş mantığı xUnit framework'ü kullanılarak test edilmiştir. Toplam 28 test yazılmış ve tamamının başarılı olduğu doğrulanmıştır.

Test Sınıfı	Test Sayısı	Test Edilen İş Kuralları
SiparisBuilderTests	6 Test	Normal sipariş akışı, stok kontrolü, ürün/ödeme doğrulaması, toplam hesabının doğruluğu.
StatePatternTests	9 Test	Durum geçişleri (Beklemede→Onaylandı vb.), kargodaki siparişe iptal engeli, iade kuralları, JSON sonrası durum yükleme.
ObserverTests	7 Test	Eşik aşılmınca bildirim, eşik aşılmayınca bildirim gitmemesi, çoklu gözlemci, gözlemci çıkarma, stok güncelleme.
DecoratorTests	8 Test	Temel kargo fiyatı, sigorta eklenmesi, kırılğan koruma ücreti, ikili decorator zincirleme, Factory Pattern doğrulama.
TOPLAM	28 Test	28/28 Başarılı — Hiç hata yok

Not: Test sonuçlarının ekran görüntüleri bu raporun ekinde yer almaktadır.